```
"""
DB-Dicionário — Backend USSD
Compatível com o formato de webhook da
Africa's Talking.

Fluxo de menu:
  Tela 0 (raiz):
    CON Bem-vindo ao DB-Dicionário!
    1. Inglês > Português
    2. Português > Inglês
    3. Palavra do dia
    0. Sair

  Escolha 1 ou 2 -> pede a palavra
  Escolha 3      -> devolve palavra
aleatória e encerra
  Depois de mostrar resultado -> END
(sessão fecha, como é padrão em consulta
única)

Toda resposta respeita o limite de 160
caracteres.
"""

from flask import Flask, request
import sqlite3
import random
import os
import requests
```

```
app = Flask(__name__)
DB_PATH = "dicionario.db"
MAX_LEN = 160

# Chave da API do Grok (xAI). Nunca escreva
a chave direto aqui —
# ela vem de uma variável de ambiente
configurada no Render (ver README).
GROK_API_KEY =
os.environ.get("GROK_API_KEY")
GROK_API_URL =
"https://api.x.ai/v1/chat/completions"


def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn


def truncar(texto: str) -> str:
    """Garante que a resposta nunca
ultrapasse o limite USSD."""
    return texto if len(texto) <= MAX_LEN
else texto[: MAX_LEN - 3] + "..."


def buscar_en_pt(palavra: str):
    conn = get_conn()
    cur = conn.cursor()
    cur.execute(
        "SELECT * FROM words WHERE
lower(word_en) = lower(?) LIMIT 1",
(palavra.strip(),)
```

```
    )
    row = cur.fetchone()
    conn.close()
    return row


def buscar_pt_en(palavra: str):
    """Busca aproximada dentro do campo
trad_pt (pode ter várias glosas separadas
por vírgula)."""
    conn = get_conn()
    cur = conn.cursor()
    termo = palavra.strip().lower()
    cur.execute("SELECT * FROM words")
    resultado = None
    for row in cur.fetchall():
        glosas = [g.strip().lower() for g
in row["trad_pt"].replace("-", "
").split(",")]
        if termo in glosas or any(termo ==
g for g in glosas):
            resultado = row
            break
    conn.close()
    return resultado


def buscar_via_grok(palavra: str, sentido:
str) -> str | None:
    """
    Fallback: se a palavra não está na base
local, pergunta ao Grok.
    sentido = 'en-pt' ou 'pt-en'. Retorna
None se a chave não estiver
```

```
    configurada ou se a chamada falhar
(nunca derruba o app).
    """
    if not GROK_API_KEY:
        return None

    if sentido == "en-pt":
        prompt = (
            f"Traduza a palavra em inglês
'{palavra}' para português. "
            "Responda SOMENTE no formato:
pronuncia_aproximada = traducao "
            "(máximo 15 palavras, sem
explicações)."
        )
    else:
        prompt = (
            f"Traduza a palavra em
português '{palavra}' para inglês. "
            "Responda SOMENTE no formato:
traducao (pronuncia_aproximada) "
            "(máximo 15 palavras, sem
explicações)."
        )

    try:
        resp = requests.post(
            GROK_API_URL,
            headers={
                "Authorization": f"Bearer
{GROK_API_KEY}",
                "Content-Type":
"application/json",
            },
```

```
            json={
                "model": "grok-4",
                "messages": [{"role":
"user", "content": prompt}],
                "max_tokens": 40,
            },
            timeout=5,
        )
        resp.raise_for_status()
        texto = resp.json()["choices"][0]
["message"]["content"].strip()
        return texto
    except Exception:
        return None


def palavra_do_dia():
    conn = get_conn()
    cur = conn.cursor()
    cur.execute("SELECT * FROM words ORDER
BY RANDOM() LIMIT 1")
    row = cur.fetchone()
    conn.close()
    return row


@app.route("/", methods=["GET"])
def health():
    """Rota simples para o Render confirmar
que o serviço está de pé."""
    return "DB-Dicionario online.", 200


@app.route("/ussd", methods=["POST"])
```

```
def ussd():
    # Parâmetros padrão enviados pela
Africa's Talking
    session_id =
request.values.get("sessionId", "")
    service_code =
request.values.get("serviceCode", "")
    phone_number =
request.values.get("phoneNumber", "")
    text = request.values.get("text", "")

    partes = text.split("*") if text else
[]
    nivel = len(partes) if text else 0

    # Tela raiz
    if text == "":
        response = (
            "CON Bem-vindo ao DB-
Dicionario!\n"
            "1. Ingles > Portugues\n"
            "2. Portugues > Ingles\n"
            "3. Palavra do dia"
        )

    # Escolheu opção do menu principal
    elif nivel == 1:
        escolha = partes[0]
        if escolha == "1":
            response = "CON Digite a
palavra em ingles:"
        elif escolha == "2":
            response = "CON Digite a
palavra em portugues:"
```

```
        elif escolha == "3":
            row = palavra_do_dia()
            if row:
                resp = f"END
{row['word_en']} ({row['pron']}) =
{row['trad_pt']}"
            else:
                resp = "END Erro ao buscar
palavra do dia."
            response = truncar(resp)
        else:
            response = "END Opcao invalida.
Tente novamente."

    # Digitou a palavra a buscar
    elif nivel == 2:
        escolha, termo = partes[0],
partes[1]
        if escolha == "1":
            row = buscar_en_pt(termo)
            if row:
                resp = f"END
{row['word_en']} ({row['pron']}) =
{row['trad_pt']}"
            else:
                grok_resp =
buscar_via_grok(termo, "en-pt")
                if grok_resp:
                    resp = f"END {termo} =
{grok_resp}"
                else:
                    resp = f"END '{termo}'
nao encontrada. Tente outra palavra."
        elif escolha == "2":
```

```
            row = buscar_pt_en(termo)
            if row:
                resp = f"END
{row['trad_pt']} = {row['word_en']}
({row['pron']})"
            else:
                grok_resp =
buscar_via_grok(termo, "pt-en")
                if grok_resp:
                    resp = f"END {termo} =
{grok_resp}"
                else:
                    resp = f"END '{termo}'
nao encontrada. Tente outra palavra."
        else:
            resp = "END Sessao invalida."
        response = truncar(resp)

    else:
        response = "END Sessao encerrada."

    return response, 200, {"Content-Type":
"text/plain"}


if __name__ == "__main__":
    port = int(os.environ.get("PORT",
5000))
    app.run(host="0.0.0.0", port=port)
```